硬件没到，怎么验证代码对不对

> 只有树莓派、没有电机的情况下，能验证到什么程度？ **答：逻辑层和接口层可以 100% 验证；物理层一个都验证不了。**

一条命令

```
ssh raspi
cd ~/four_car_ws
bash test_no_hardware.sh
```

跑完会给一个总结。**当前实测：15 / 15 全部通过。**

这套测试之所以能测，是因为代码从一开始就分成三层：

```
硬件驱动层    ← 只有这层碰真东西（没有硬件 = 这层是空的）
逻辑算法层    ← 纯数学，用假数据就能测
ROS 节点层    ← 只调接口，仿真的假的都一样
```

15 项测试，分别证明了什么

#	测试	证明了	证明不了
1	`selftest.py` 29 项	麦轮正逆解算公式自洽、限幅保方向、里程计积分正确、卷尺时间估算合理	**符号约定对不对** —— `flip_vy` / `flip_wz` 只有实车往哪走才能验
2	四个节点启动	代码没有语法/导入/参数类型错误	节点能不能真的驱动硬件
3	`cmd_vel → odom`	订阅回调通、麦轮逆解算通、里程计积分通、消息类型对	电机真的转了
4	`arm_cmd → DOCKED`	上层状态机流转正确、话题接口通	卷尺真的伸出来了
5	`/mission/state → MOVING`	状态机能被外部触发、状态切换正确	—
6	参数列出/修改	参数声明正确、在线调参可用	参数真的影响了硬件行为
7	仿真全流程	**4 车分流、集齐才推进、超时兜底、周期重发** 这套编排逻辑全对	真实时序够不够（仿真里 3 米是瞬间算出来的）

#	测试	证明了	证明不了
8	缺串口报错	硬件缺失时能给出可操作的提示，不是一屏 traceback	—

> **最有价值的是第 7 项**

因为它验证的是**整个系统最难写对的部分**：4 台车各自的节奏不一样，怎么保证"都到齐了才进下一步"。这个逻辑跟硬件完全无关，仿真能 100% 验证。

```
真实输出：
  === 广播 GO ===
  car3/car4: READY -> TURNING -> ALIGNED      ← 后两台不直行，先到位
  car1/car2: READY -> MOVING -> TURNING -> ALIGNED
  （集齐 4 个 ALIGNED）
  === 广播 EXTEND ===
  4 台车: ALIGNED -> EXTENDING -> DOCKED
  （集齐 4 个 DOCKED）
  === DONE ===
```

★ 测不了的（必须等硬件）

这一节比上面重要 —— **别把"15/15 通过"误当成"东西没问题了"。**

测不了	为什么	硬件到了之后怎么测
麦轮方向对不对	数学自洽 ≠ 车往对的方向走。辊子朝向决定符号	使用手册 阶段 E：发横移看往哪走
四个轮子装反没	同上	使用手册 阶段 E：发前进看轮子转向
串口协议通不通	`ZDTBus` 发出去的字节，电机到底认不认	阶段 C：`zdt_id_tool.py scan`
电机地址是几	出厂全是 1，四个都是 1 就乱套	阶段 B：`zdt_id_tool.py setup`
减速比设对没	这是**驱动器内部**的设置，代码只能读不能替它验证	阶段 C：`zdt_id_tool.py gear`
卷尺伸出要几秒	电池电压、机构阻力、卷尺松紧都影响	阶段 F：从 3 秒开始加
电磁铁极性	MOSFET 电路反相的话，通电反而不吸	阶段 G：发 DOCK 看吸不吸
L298N 跳线帽拔没	插着的话 PWM 静默失效（电机只会全速）	阶段 B：空载看电机转速变不变
共地了没	不共地的症状千奇百怪	阶段 A：万用表量通断

测不了	为什么	硬件到了之后怎么测
4 台车跨机通信	现在 4 台车跑在**同一台树莓派**上。跨 WiFi 的丢包、延迟、DDS 发现**完全没验过**	阶段 I：4 台真机一起跑

> ⚠️ **最后一条是整个项目最大的未验证风险。** 单机仿真跑得再顺，跨 4 台树莓派的 WiFi 通信是另一回事（丢包、时钟、DDS 发现）。硬件到了要**优先验证这一项**。

各项测试的真实输出（对照用）

跑 `test_no_hardware.sh` 时你应该看到这些。对不上就是有问题。

0. 环境

```
[OK]   ROS_DISTRO=jazzy
[OK]   工作空间已加载
[OK]   gpiozero 已装（真机 GPIO 需要）
```

1. 纯逻辑

```
[OK]   selftest 通过（29 项）
```

2. 节点启动

```
[OK]   base_node 启动正常
[OK]   arm_node 启动正常
[OK]   agent_node 启动正常
[OK]   mission_node 启动正常
```

单独跑的时候，日志应该是：

```
# ros2 run car_base base_node --ros-args -p simulate:=true
[base_controller]: MecanumKinematics(r=0.0485, Lx=0.0801, Ly=0.1385, L=0.2186, flip_vy=False, flip_wz=False)
[base_controller]: 底盘就绪 | 仿真 | 控制 30Hz 里程计 20Hz | cmd_vel 超时 0.5s 自动停车

# ros2 run car_arm arm_node --ros-args -p simulate:=true
[arm_controller]: 上层就绪 | 俯仰 +0.0° | 伸出时间 8.0s | 【仿真】

# ros2 run car_agent agent_node --ros-args -p simulate:=true
[car_agent]: car1 就绪 | 目标航向 +90° | 先直行 3.00m | 【仿真模式】
```

```
# ros2 run mission_controller mission_node --ros-args -p autostart:=false
[mission_controller]: 编排层就绪 | 车辆 [1, 2, 3, 4] | 配对 [[1, 2], [3, 4]] | 超时 30s | 等待手动触发
```

3. 接口连通

```
[OK]   cmd_vel -> odom 有反应（x=1.5750257870000002）
[OK]   arm_cmd EXTEND -> 走到 DOCKED
[OK]   /mission/state GO -> READY->MOVING
```

> 那个 `x=1.575` 是 0.5 m/s × 3.15 秒，数值对得上就说明里程计积分正确。

4. 参数

```
[OK]   参数可列出（23 个）
[OK]   参数可在线修改
```

5. 完整流程

```
[OK]   仿真全流程走通（GO -> EXTEND -> DONE）
```

6. 缺硬件时的报错

```
[OK]   缺串口时报错清晰
       [FATAL] [base_controller]: 底盘初始化失败：打开电机串口 /dev/ttyUSB0 失败: ...
         当前存在的串口: /dev/ttyAMA0, /dev/ttyAMA10
         · 底盘用 USB 那个（ttyUSB*/ttyACM*）
         · /dev/ttyAMA0 是**舵机**用的，别占它
       [FATAL] [base_controller]: 节点退出。想先不接硬件调试的话，加 -p simulate:=true
```

这条尤其值得看：它列出了机器上现在实际有哪些串口，硬件到了插上就能一眼看出该用哪个。

自己手动一个个测（不想跑脚本）

```
source /opt/ros/jazzy/setup.bash
source ~/four_car_ws/install/setup.bash
export ROS_DOMAIN_ID=0
```

```
# ① 纯逻辑
cd ~/four_car_ws && python3 selftest.py

# ② 节点能不能起（终端 1）
ros2 run car_base base_node --ros-args -p simulate:=true

# ③ 接口通不通（终端 2）
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"
# 终端 3 看反应：
ros2 topic echo /odom

# ④ 上层（终端 1 换成 arm_node 后）
ros2 topic pub --once /arm_cmd std_msgs/msg/String "{data: 'EXTEND'}"
ros2 topic echo /arm_state

# ⑤ 完整流程
ros2 launch four_car_bringup sim.launch.py

# ⑥ 想看 4 台车在干什么（最直观）
ros2 topic echo /rosout
```

想加新测试怎么办

`test_no_hardware.sh` 就是个普通 bash 脚本，加一项：

```
head_ "7. 我要测的新东西"
if <某个条件>; then
    ok "说明"
else
    bad "说明"
fi
```

`ok` / `bad` 会自动计数，最后一起算总结。

什么时候算"代码没问题了"

阶段	判据
现在（硬件没到）	`test_no_hardware.sh` 15/15，且 `sim.launch.py` 能走到 DONE
硬件到了（还没装车）	使用手册 阶段 A~D：串口通、能扫到电机、单个轮子能转
装车后	阶段 E~G：麦轮方向标定、卷尺时间定下来、磁铁吸合正常
单车完整	阶段 H：`car.launch.py` 实机跑通一个完整任务
真正能用	阶段 I：**4 台树莓派跨机跑通** ← 这才是最终关卡